The B2I ratio

To calculate the B2I ratio, determine the number of lines of code (LOC) before adding an instrumentation (adding code for emitting signals for a signal type), and then determine the LOC after the instrumentation. The B2I ratio is then

```
B2I = LOC_AFTER_INSTRUMENTATION/LOC_BEFORE_INSTRUMENTATION
```

In an ideal world, the B2I ratio would be 1, representing zero instrumentation costs in the code. However, in reality, the more LOC you dedicate to instrumentation, the higher the B2I ratio is. For example, if your code has 3800 LOC and you added 400 LOC for instrumentation (say, to emit logs and metrics), then you'd end up with a B2I ratio of 1.105, from (3800 + 400) / 3800.

There are two things worth mentioning concerning B2I:

- It's up to you to decide whether to consider telemetry libraries such as an OpenTelemetry SDK (https://opentelemetry.io/docs/instrumentation/) in your B2I ratio calculation. From a pragmatic point of view, focusing on your own code (excluding these dependencies) might make the most sense. This is what I will do in the remainder of the chapter.
- The B2I ratio is a simple measure and should by no means serve as an indicator of how well your code is instrumented or tell you if your code is *over-instrumented*. Goodhart's law (see Strathern and Marilyn, 1997, "Improving Ratings: Audit in the British University System," *European Review*, John Wiley & Sons) states that when a measure becomes a target, it ceases to be a good measure—this applies here. Use the B2I ratio as a rough guide in a regression validation, rather than an absolute North star.

With the theory and overview out of the way, let's now review the three basic signal types: logs, metrics, and traces. In the remainder of the chapter, we will do the following for each signal type:

1. Define the signal characteristics and semantics.
2. Discuss how to instrument our example `echo` service.
3. Set up the telemetry to collect and route the signal.
4. Analyze costs (in terms of resource usage and efforts) and benefits of the signal.
5. Explain how the respective signal can contribute to an observability solution.

2.3 Logs

Let's begin by focusing on logs. Logs are signals with a textual payload that capture an event. An *event* is an umbrella term for anything of interest that happens in the code, be it on the happy path (a completed upload) or unexpected or erroneous behavior ("failed to store items in shopping basket").